GeoMind 技术手册

AI 竞赛几何工作台：架构、功能与实现逻辑

2026-10-22

0. 设计目标与原则

GeoMind 的目标不是 “生成一张看起来像题的图”，而是把平面几何对象变成**可计算**、**可拖动**、**可验证**、**可导出**的结构。

四条原则：

1. **对象优先**：画板保存对象与依赖，不保存最终像素。
2. **尺规优先**：能用派生构造表达的，不用自由点或数值拟合。
3. **验证优先**：任何目标都先算残差，再谈证明。
4. **失败可见**：识图/求解失败必须进入日志，不能静默标为正确。

---

1. 总体架构

```
Browser
  index.html / css
  js/app.js          UI、Renderer、交互、Algebra、Script、Vision 安装
  js/geometry.js     几何对象、依赖图、构造、重算
  js/relations.js    关系枚举与评分
  js/solver.js       目标求值、稳定性、证明叙事
  js/constraints.js  数值约束求解
  js/complex.js      复平面复验
  js/vision.js       识图 JSON -> construction -> board
  js/scenarios.js    回归构型
        |
        | HTTP JSON
        v
```

```
Python server.py
  validation / generation / vision / proof jobs / logs / PDF
        |
        +-- OpenCode 多模态模型（识图）
        +-- mini-harness agent（纯几何证明）
        +-- ctexart + xelatex（PDF）
```

前端是原生 ES modules，没有构建步骤；Canvas 2D 负责绘图；Web Worker 负责关系扫描。后端是 Python 标准库 HTTP 服务，mpmath 做 50 位十进制验证，Pillow 处理图，XeLaTeX 编译 PDF。

---

2. 状态模型与持久化

`createState()` 生成：

```
{
  version: 1,
  docName,
  objects: [],
  counters: {},
  view: { cx, cy, scale },
  settings: { grid, axes, labels, snap, wheel, theme, ... },
  meta: { createdAt, revision, scenarioId }
}
```

对象统一结构：

```
{
  id, type, sub, label, deps,
  visible, locked, style, meta, g, valid, error
}
```

- `deps`：依赖对象 id，是构造图的边；
- `g`：本次重算得到的几何数据，不序列化；
- `valid`：本次计算是否有效；
- `style`：颜色、线宽、虚线、填充、点大小、文本字号；
- `meta`：辅助点、中心颜色、动态约束、标签位置等。

`serializeState` 去掉 `g/valid/error/metrics`，保留依赖与样式；`deserializeState` 恢复后立即 `recompute`。

---

3. 依赖图与重算

3.1 建图与拓扑排序

`recompute(state)`：

1. 用 `deps` 建图；
2. 拓扑排序，保证依赖先算；
3. 对每个对象调用 `computeObject(o, byId)`；
4. 写入 `o.g`、`o.valid`、`o.error`；
5. 统计耗时。

循环依赖不会被计算，对象 `valid=false`。

3.2 无效传播

任何依赖无效，当前对象直接无效。比如圆-线交点：

```
line.valid && circle.valid
  -> intersectionOfCurves
  -> 无交点时 valid=false
```

这样派生点不会悄悄落到错误位置。

3.3 重算触发

以下操作会触发：

- 拖动点；
- 修改属性；
- 新建/删除对象；
- 载入 scenario/JSON；
- 脚本执行；
- 识图安装；
- 约束求解每次迭代。

4. 几何内核

4.1 向量与数值

geometry.js 内部统一用：

dot, cross, add, sub, mul, dist, norm, mid, angleAt, ...

阈值 EPS 用于退化判断。角度使用有向角：

- directedAngle2：两方向；
- directedAngle3：三点；
- mod 180 或 mod 2 按用途选择。

4.2 点构造

pointG(o, byId) 的核心分支：

sub	含义	依赖
free	自由点	无
pointOnLine	线上/线段上点	line/segment + t
pointOnCircle	圆上点	circle + theta
intersection	线/圆交点	两曲线 + branch
midpoint	中点	两点
foot	垂足	点 + 线
midpointArc	弧中点	圆 + 弧端点 + 过点
incircleTouch	内切圆切点	三角形三顶点
circumcenter	三点外心	三点
triangleCenter	三角形中心	triangle + centerKey
reflectLine / reflectPoint	反射	点 + 线/点
rotate / translate / homothety	变换	点 + 参数
inversion / iranianInversion	反演	点 + 圆
tangentPoint / tangentCirclePoint	切点	圆/线
tangencyPoint	两圆切点	两圆 + branch
angleDouble	角二倍条件点	线段 + 角点
equalDistance	等距点	U,B,C,A
ratioDistanceOnLine	比例点	X,B,C,A
pointOnRayDistance	射线上定长点	A,B,C,D
parallelogramFourth	平行四边形第四点	A,B,C
angleEqualOnLine	线上等角点	线 + 四点
angleAtFixedOnLine	线上固定角点	线 + 五点

4.2.1 弧中点

`midpointArc` 根据依赖判断：

- 4 个依赖 `[圆,B,A,C]`：取含 A 的弧 BAC 中点；
- 3 个依赖 `[圆,B,C]`：取另一段弧 BC 中点。

实现上计算 B、C 相对圆心的角，取两段弧的中点，再按 A 所在弧选择。

4.2.2 内切圆切点

```
a = |BC|, b = |CA|, c = |AB|, s=(a+b+c)/2
AF = s-a, BF = s-b
F = A + (AF/|AB|)(B-A)
```

4.2.3 比例点

`ratioDistanceOnLine(X,B,C,A)`：

```
len = |BX| * |AC| / |AB|
Y = C + unit(C-B) * len
```

用于 `BX · AC=CY · AB` 这类条件。

4.2.4 切点

`tangentAtPoint(A,B,line)` 求过 A、B 且与给定直线切于 A 的圆：

- 圆心在 A 的法线上；
- 圆心到 A、B 等距；
- 解一元方程得到圆心和半径。

`tangencyPoint(c1,c2)` 求两圆切点：

```
u = unit(O2-O1)
P = O1 + u * r1
```

branch 控制内切/外切方向。

4.3 线构造

sub	含义
`through`	过两点

sub	含义
`parallel`	过点平行基线
`perp`	过点垂直基线
`perpbis`	两点中垂线
`bisector` / `externalBisector`	角平分线/外角平分线
`bisector2`	两线角平分线
`isogonal`	等角线
`tangentAt`	圆上一点的切线
`tangentFrom`	圆外一点的切线
`radical`	两圆根轴
`altitude` / `median` / `euler` / `simson`	三角形特殊线
`fixedAngle` / `fixedAngle3`	定角线
`polar`	极点极线
`inversionOfLine`	线的反演像

4.4 圆构造

`circleG`：

- `threePoints`：三点外接圆；
- `centerThrough`：圆心 + 过点；
- `diameter`：直径圆；
- `incircle` / `excircle`：内切/旁切圆；
- `ninePoint` / `triangleNinePoint`：九点圆；
- `tangentAtPoint`：过 A、B 且切定线于 A；
- `radiusFrom`：半径由长度对象决定；
- `centerRadius`：圆心 + 固定半径；
- `inversionOfCircle`：圆的反演像。

4.5 曲线

`curve` 支持：

- 五点 conic：`conicValue` 行列式；
- QL-Cu1：pedal concyclicity 行列式；
- marching squares 采样成线段，供渲染和命中。

曲线对象只用于显示/近似交点，不参与严格 symbolic 证明。

5. 渲染器

`Renderer` 持有 canvas、state、DPR 缩放。

5.1 坐标变换

```
toScreen(p) = {
  x: view.cx + p.x * view.scale,
  y: view.cy - p.y * view.scale
}
```

y 轴翻转，保证数学坐标 y 向上。

`toWorld` 做逆变换。

5.2 绘制顺序

网格/坐标轴
曲线
线/射线/线段
圆/弧
多边形/三角形填充
角标记
点
测量
文本
选中高亮/拖拽预览

每帧根据 `dirty` 重绘；`requestAnimationFrame` 合并高频请求。

5.3 标签避让

点标签在点右上方；中心点用中心色；文本/测量标签可被拖动，测量对象保存 `meta.labelPos`。

5.4 命中测试

`hitTest(screen)` 顺序：

1. 点（半径随触摸放大）；
2. 测量标签（优先于下方线段）；
3. 线/线段/射线（屏幕距离）；

4. 曲线采样段；
5. 圆/弧（径向距离）；
6. 多边形/文本。

拖动测量/文本标签使用独立的 `labelDrag` 状态，不混入点拖动。

---

6. 交互与 UI

6.1 工具系统

`TOOL_GROUPS` 定义工具组：

- 基本：选择、点、线上点、线段、直线、射线、向量、圆、三点圆、直径圆、多边形、文本；
- 构造：中点、垂足、交点、平行、垂直、中垂线、角平分线、切线、外心/内心/旁心/垂心/重心/九点圆心、定长、定角、等距点；
- 变换：旋转、平移、位似、反射、反演；
- 射影：极点极线、交比、调和点；
- 度量：距离、角度、面积、比例；
- 对象：对象列表。

多步工具使用 `S.ui.pending` 记录已选对象；`finishConstruction` 清空并选中结果。

6.2 拖动

`isDraggablePoint` 只允许：

`free / pointOnCircle / pointOnLine`

且`!locked`。拖动时：

- `pushHistory()`；
- 记录起始点；
- pointermove 更新坐标/参数；
- `recompute` 全部依赖；
- pointerup 提交历史。

`locked` 用于约束点、固定点、P/Q 等条件点。

6.3 移动端

媒体查询 <=940px：

- 右侧 Inspector 默认隐藏；
- 左侧 rail 保留；
- 工具列表变成底部 sheet；
- 对象列表变成左侧抽屉；
- 顶部工具栏可关闭；
- 文本/测量标签可拖动。

---

7. 关系发现

`buildSnapshot(state)` 抽取：

- 点坐标、标签；
- 线方向/端点；
- 圆圆心/半径；
- 三角形中心；
- 曲线采样。

`runDetection(snapshot, options)` 枚举关系：

- 共线：三点叉积；
- 共圆：四点行列式；
- 平行/垂直：方向叉积/点积；
- 等长/等角/中点/相切/比例；
- 特殊定理：欧拉线、九点圆、西姆松线等；
- 评分：残差 + 稳定性 + 特殊度。

关系扫描在 Web Worker 中执行；失败回退主线程。

---

8. 目标求值

`evaluateGoal(state, goal)` 返回：

```
{ ok, residual, threshold, values, detail, stable }
```

每个目标做归一化残差：

- 共线：叉积 / 尺度；
- 共圆：行列式 / 尺度 ^4；
- 垂直：单位方向点积；

- 平行：单位方向叉积；
- 相切：圆心到线距离 - 半径，或圆心距 vs 半径和/差；
- 等长：相对差；
- 等角：角度差 / ；
- 角度和：和 - 目标；
- 共点：三线到公共点最大距离 / 尺度。

`stabilityCheck` 对自由点做小扰动，观察残差是否仍小，用于标记 “偶然成立”。

---

9. 通用约束求解

`constraints.js`：

9.1 未知量

- 自由点 x/y；
- `pointOnLine.t`；
- `pointOnCircle.theta`。

9.2 残差

支持：

`equalLength, equalAngle, angleDouble, collinear, concyclic, parallel, perpendicular, tangent, midpoint, onLine, onCircle, ratio, circlesTangent, concurrent`

残差统一除以当前尺度，便于和阈值比较。

9.3 求解

有限差分 Jacobian：

```
J_ij = (r_i(x+h_j) - r_i(x)) / h
```

解阻尼正规方程：

```
(J^T J +  I) dx = -J^T r
```

线搜索接受下降步；失败则回退。求解后仍走 `recompute` 和目标验证。

9.4 尺规优先

construction 优先使用派生点；只有无法尺规表达时才使用 solver。校验会标记 “有约束但仍是 free/pointOnLine” 的点。

---

10. 脚本系统

10.1 DSL

逐行解释：

```
A=(x,y)
Segment(A,B)
Line(A,B)
Circle(A,B)
Midpoint(A,B)
Intersect(a,b)
PerpendicularLine(P, line)
ParallelLine(P, line)
Polygon(...)
Distance(...)
Angle(...)
Sequence(...)
SetColor / SetVisible / SetValue / Delete
ZoomIn / ZoomOut
```

解释器基于递归下降表达式 `mathEval`，共享几何依赖图。

10.2 JavaScript

`runJsScript` 提供：

```
api.point(label,x,y)
api.segment(a,b)
api.line(a,b)
api.circle3(a,b,c)
api.arcMid(...)
api.incircleTouch(...)
api.midpoint/foot/intersection
api.rayDistance(...)
```

```
api.bisect(f,lo,hi)
api.solve(constraints)
api.goal(goal)
api.recompute()
```

执行前 pushHistory；执行后 afterMutation。脚本在浏览器内运行，不访问网络。

---

11. 识图管线

11.1 请求

POST /api/vision 接收：

```
{ "image": "...", "text": " 题目文字" }
```

后端只调用多模态模型，不做本地 OCR/规则回退。

11.2 返回结构

```
{
  "title": "...",
  "problem_text": "...",
  "triangle": {...},
  "points": [...],
  "segments": [...],
  "lines": [...],
  "circles": [...],
  "constraints": [...],
  "construction": [...],
  "script_js": "...",
  "goal": {...}
}
```

11.3 坐标

图片坐标归一化到 0..1，worldFromNorm 映射到画板：

```
x = (nx - .5) * 11
y = (.5 - ny) * 9.5
```

自由点优先 `at`，回退 `points[].coordinate`。

11.4 construction 执行

`buildConstructionState` 多轮迭代：

1. 自由点 / 线 / 圆；
2. 依赖满足后建派生点；
3. 未解析的 `pointOnLine` 用原图坐标回退为自由点；
4. 交点按原图坐标选 branch；
5. 执行 constraints 求解；
6. 生成 goal。

11.5 修复

`validateVisionState` 检查：

- 点是否存在/valid；
- 自由点是否偏离原图；
- 目标是否生成；
- 有约束点是否错误写成自由点。

失败会写后端 `vision_check` 日志，并把错误反馈模型修复一次。

---

12. 纯几何证明

12.1 任务

`POST /api/proof/start` 创建异步任务；`/api/proof/status?job=` 轮询。

12.2 mini-harness

证明走 proj05 mini-harness：

- 读取 `~/.dsh/skills/pure-geometry/SKILL.md`；
- 截取操作规则注入 system prompt；
- `POST /v1/agent/run`；
- agent 可用沙箱工具；
- 返回 JSON 证明。

12.3 PDF

`_compile_proof_pdf`：

- ctexart；
- 题目 + 证明 LaTeX；
- 默认附画板截图；
- `PROOF_TIKZ=1` 才使用模型 TikZ；
- xelatex 编译，失败回退纯文本证明。

---

13. 复数法

`complex.js` 负责前端数值复验；新增 `symbolic_geometry.py` 提供后端符号编译器。

符号编译器把自由点设为实数符号 (`x_A, y_A`)，把中点、线上点、线线/线圆交点、圆上点、反射、三点外心、三角形 O/G/H/N 等编译成 SymPy 表达式，再把目标化为复比/交比/模长恒等式，用 `simplify/together` 判定是否为零。

端点：`POST /api/complex/symbolic`，输入序列化画板 `state` 与 `goal`。前端「符号推导恒等式」按钮调用它。

支持的 AI 目标：

`共线 / 共圆 / 垂直 / 平行 / 等长 / 等角 / 角度和 / 相切`

UI 不再输入 `coll(...)`，直接读取 `S.ui.goal`。

---

14. 导出

- JSON：完整依赖 + 样式；
- PNG：Canvas 当前视图；
- SVG：由对象生成 SVG；
- TikZ：点、线、圆；
- PDF：Canvas PNG 上传后端，ctexart/xelatex 编译。

---

15. 日志

- 本地 `S.ui.actionLog`：构造、载入、脚本；
- 后端 `/api/logs`：vision、proof、proof_job、vision_check；
- 识图校验失败会显示具体点；
- 不记录 API Key 和完整图片。

---

16. 配置与安全

- 后端默认模型 Key 只在服务端；
- mini-harness Key 在`.secrets/miniharness_key`，权限 600；
- 公网部署需鉴权/限流；
- 日志接口不要暴露给不可信用户；
- 脚本在浏览器本地执行。

---

17. 测试与支持状态

- 关系测试：44 条；
- 动态回归构型：30 个；
- 参考 PDF 支持状态：`docs/SUPPORT_STATUS.md`；
- 100 题原图题库：`data/league100/`；
- 识图 construction / ratio / goal 测试在 `test/geometry.test.mjs`。

---

18. 设计取舍与限制

1. VLM 识图是近似识别，不是形式化理解；
2. 尺规构造语法仍有限；
3. 约束求解可能退化/无解；
4. 动态条件构型需要保留参数或重新求解；
5. 100 题规模下不能保证全部自动正确；
6. 不把失败静默标为成功。

系统的核心不是“猜对图”，而是：**可构造**、**可回算**、**可验证**、**可解释**。